



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
GRADUAÇÃO EM ENGENHARIA DE SOFTWARE

Arquitetura de Software

Aula – Apache Cassandra

Prof.: Filipe Nhimi

Agenda

- ☐ História
- ☐ Funcionamento
- ☐ Conclusão



Apache

CASSANDRA

A large, blue, three-dimensional Facebook logo is mounted on the side of a building at night. The logo is partially obscured by a dark blue rectangular box in the lower-left corner. The background shows the building's facade and some colorful lights (purple, orange, blue) reflecting on the glass windows.

2007

A história de Cassandra começou em 2007 no Facebook, onde **engenheiros buscaram um sistema que pudesse armazenar dados para a crescente plataforma de mensagens da empresa.** Combinando modelos de banco de dados NoSQL estabelecidos (Amazon Dynamo e Google Bigtable), eles **criaram um sistema com estruturas de dados eficientes e consistência eventual, em que as atualizações se propagam até que todas as réplicas correspondam com o passar do tempo.**

The Inspiration Behind Cassandra

Cassandra



Google Bigtable

Column Families

Memtables

SSTables

Amazon Dynamo

Consistent Hashing

Partitioning

Replication



Avinash Lakshman ✓

Refer to audio recording · 3°



Founder/CEO @ Weilliptic Inc. | AI, Blockchain

Palo Alto, Califórnia, Estados Unidos · [Dados de contato](#)

✉ Enviar mensagem

+ Seguir

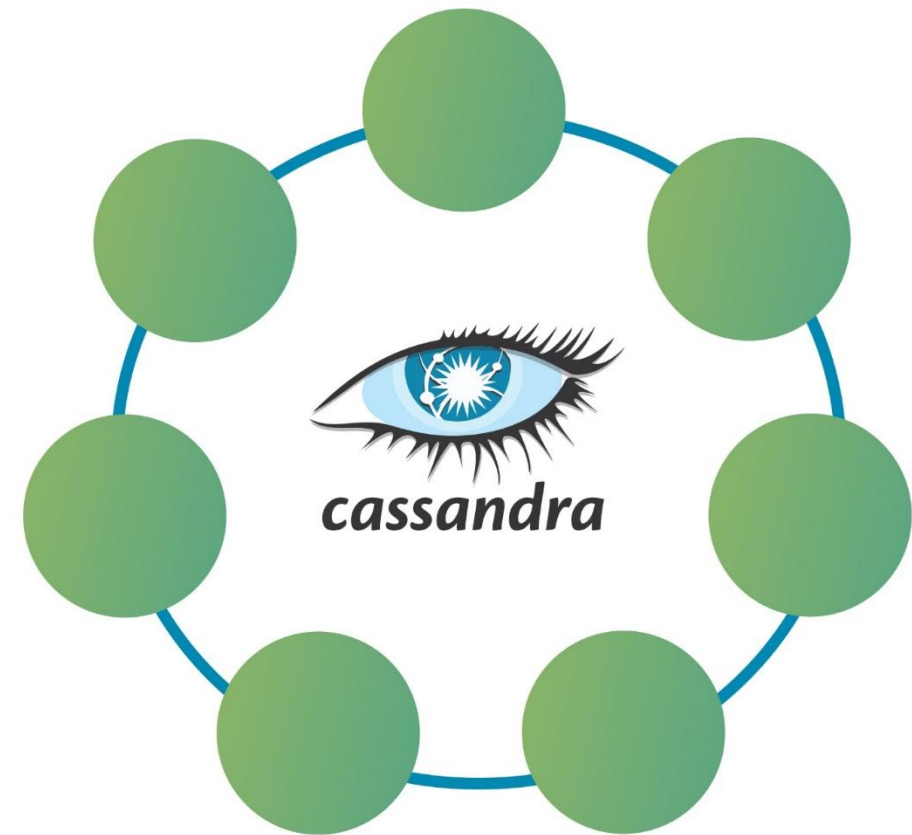
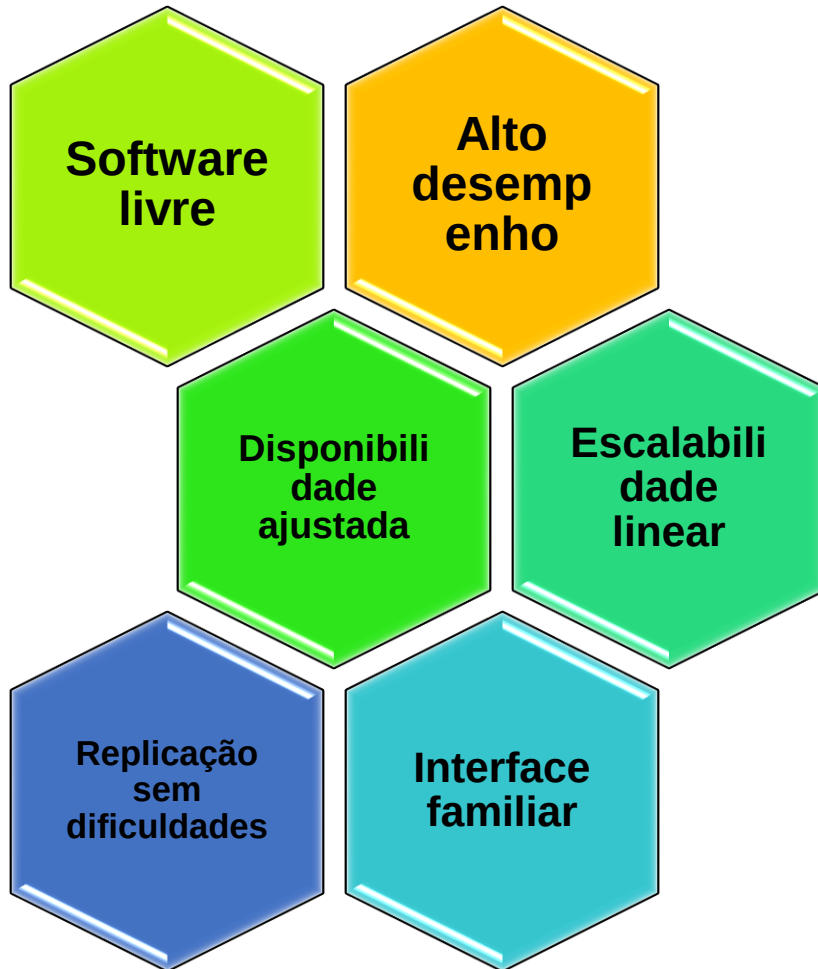
Mais

Sobre

Inventor of Cassandra
Co-inventor of Amazon Dynamo

<https://www.linkedin.com/in/avinashlakshman/>

Características principais do Cassandra



**Peer-to-Peer, não existe Master
nem Servidor Central!**

Código aberto

O Cassandra é **código aberto** sob a ***Apache Software Foundation***, ajudando as organizações a evitar o lock-in com fornecedor e personalizar o banco de dados para atender às suas necessidades. Quando a ajuda de nível empresarial é necessária, as equipes podem usar os recursos da comunidade ou escolher suporte comercial e serviços gerenciados.

Alto desempenho

O mecanismo de armazenamento do Cassandra usa um fluxo passo a passo (ou caminho de gravação) que consiste em um **log de confirmação**, uma **tabela na memória (memtable)** e **arquivos de tabela de strings classificadas** (SSTable). Esse fluxo aceita operações de gravação rapidamente e as protege. **Os dados acessados com frequência são mantidos no cache para consultas de baixa latência**, enquanto a compactação, uma função de manutenção automática, ajuda a garantir um armazenamento de dados eficiente a longo prazo.

Disponibilidade ajustada

De acordo com o teorema CAP, quando ocorre uma partição de rede, **um sistema distribuído pode oferecer apenas duas das três características desejadas: consistência, disponibilidade e tolerância de partição (CAP).** O Cassandra lida com essa desvantagem por meio de níveis de consistência ajustáveis, permitindo que os usuários priorizem a disponibilidade ou a consistência, dependendo do caso de uso.

Escalabilidade linear

O Cassandra aumenta a capacidade adicionando novos nós sem interrupção do serviço, oferecendo escalabilidade linear em servidores comuns em vez de atualizações verticais caras. **À medida que os nós são adicionados, o Cassandra redistribui automaticamente os dados e o tráfego pelo cluster** para que as cargas de trabalho se expandam e a taxa de transferência aumente proporcionalmente.

Replicação sem dificuldades

O Cassandra replica dados em nós e data centers para que os usuários locais tenham baixa latência enquanto **evitam um único ponto de falha**. Ele também se integra ao Kubernetes, às estruturas de interface de programação de aplicativos (API) e aos ambientes Amazon Web Services (AWS). É programado em Java e executado na Máquina Virtual Java (JVM).

Interface familiar

As equipes usam o **Cassandra Query Language (CQL)** — **que espelha o SQL — para definir rapidamente as construções de chaves, como keyspace, tabelas e chaves primárias.** Ferramentas interativas como o CQL shell (cqlsh) e tutoriais oficiais também podem ajudar a reduzir o tempo de integração de novos desenvolvedores.

CQL vs. SQL

Embora o CQL seja parecido com o SQL, **as duas linguagens refletem abordagens diferentes para o gerenciamento de dados.**

Estruturas de dados

O SQL opera em tabelas normalizadas, enquanto o CQL é projetado para dados desnormalizados do Cassandra alinhados com chaves de partição.

Consistência

O SQL pressupõe a integridade rigorosa dos dados, enquanto o Cassandra equilibra a consistência eventual com níveis de consistência configuráveis.

Escalabilidade

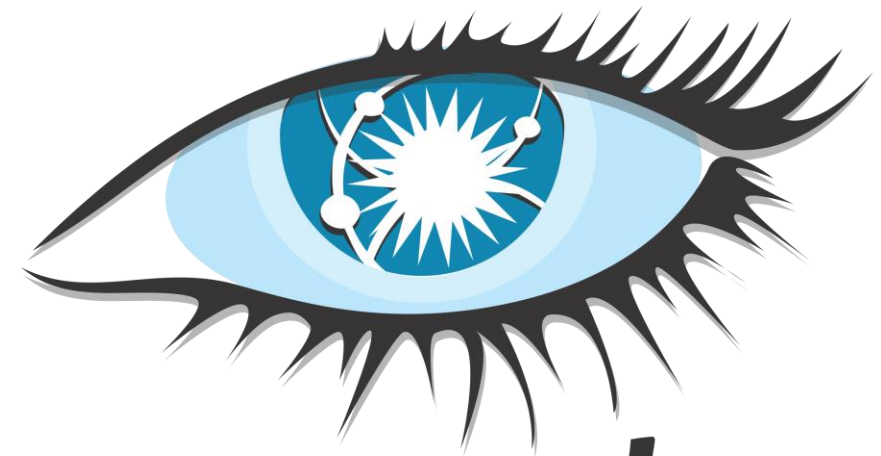
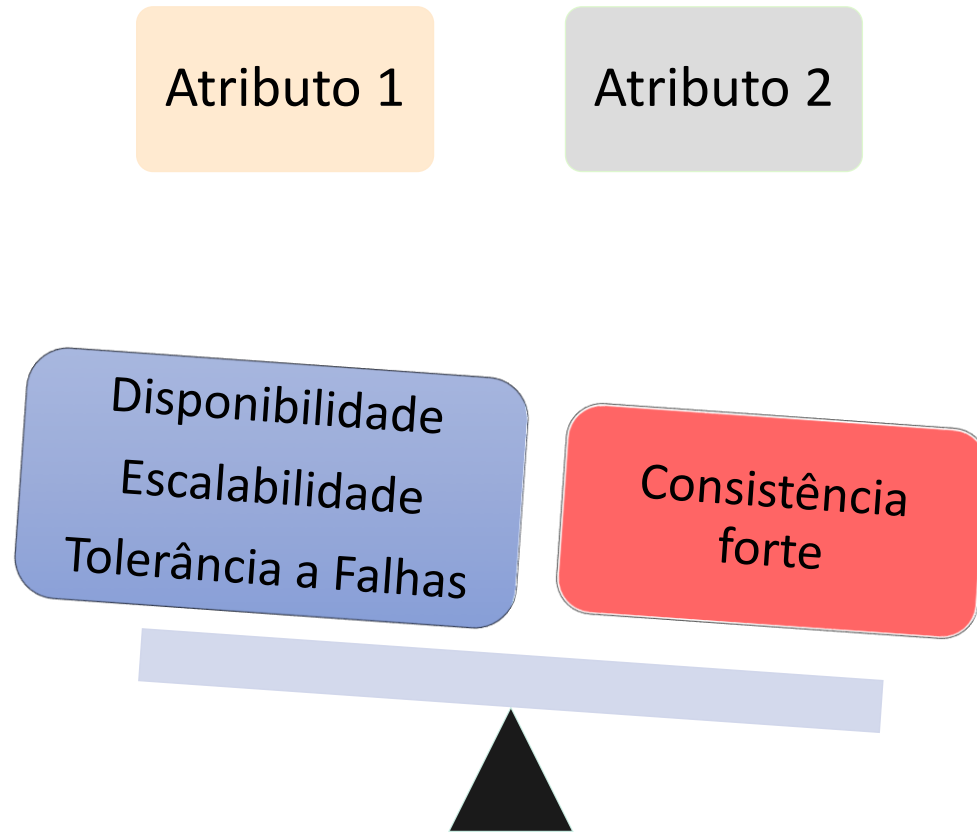
Os sistemas SQL normalmente dependem de escala vertical, enquanto o Cassandra possibilita a escalabilidade linear adicionando novos nós a um cluster.

Operações

O SQL é otimizado para transações, enquanto o CQL é projetado para consultas em tempo real e operações de gravação de alto volume.

Os desenvolvedores que migram do SQL podem se adaptar rapidamente à sintaxe do CQL, mas devem repensar as estratégias de modelagem de dados para aproveitar a abordagem de sistemas distribuídos do Cassandra.

Trade-offs



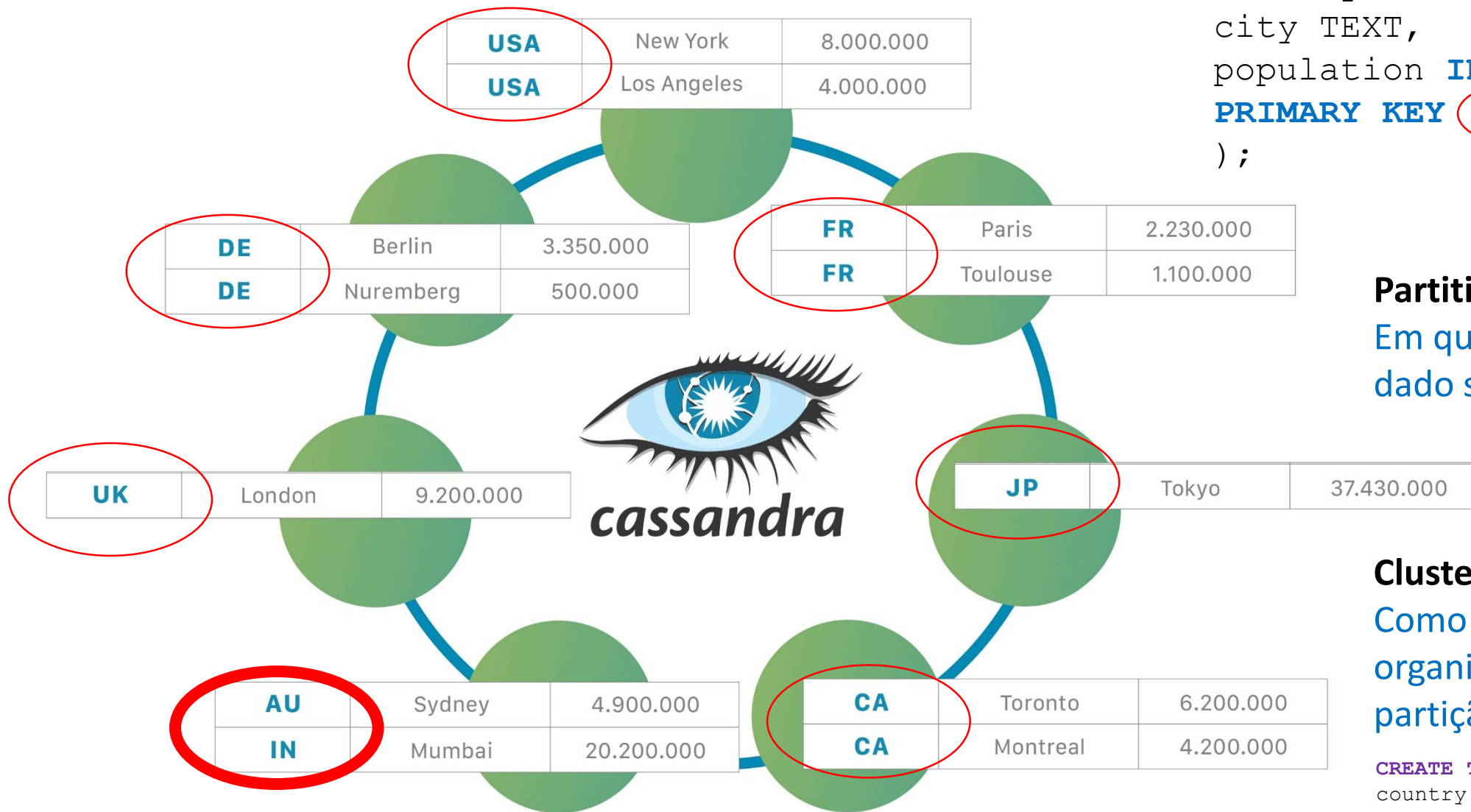
cassandra



COUNTRY	CITY	POPULATION
USA	New York	8.000.000
USA	Los Angeles	4.000.000
FR	Paris	2.230.000
DE	Berlin	3.350.000
UK	London	9.200.000
AU	Sydney	4.900.000
DE	Nuremberg	500.000
CA	Toronto	6.200.000
CA	Montreal	4.200.000
FR	Toulouse	1.100.000
JP	Tokyo	37.430.000
IN	Mumbai	20.200.000


Partition Key

https://cassandra.apache.org/_/index.html



```
CREATE TABLE cities (  
country TEXT,  
city TEXT,  
population INT,  
PRIMARY KEY (country, city)  
);
```

Partition Key = country
Em qual nó do cluster o dado será armazenado?

Clustering Key = city
Como os dados serão organizados dentro da partição? **ordenação**

opcional

Um nó normalmente armazena milhares ou milhões de partições

```
CREATE TABLE cities (  
country TEXT,  
city TEXT,  
population INT,  
PRIMARY KEY (country, city)  
) WITH CLUSTERING  
ORDER BY (data_venda DESC);
```

CASSANDRA COMO DBaaS NA NUVEM

Principais serviços gerenciados que oferecem Apache Cassandra



Amazon Keyspaces (Apache Cassandra)

Totalmente gerenciado,
compatível com Cassandra
open source.

REGIÕES PRINCIPAIS

US East (N. Virginia)
US West (Oregon)
Europe (Frankfurt)
Asia Pacific (Sydney)
e outras

DESTAQUES

- Compatível com Cassandra OSS (CQL)
- Escalabilidade automática
- Multi-AZ
- Backups automáticos
- Integração com IAM, VPC e CloudWatch

 aws.amazon.com/keyspaces



Google Cloud

Cloud Firestore in Datastore Mode (Cassandra API)

API Cassandra compatível
sobre o Firestore.

REGIÕES PRINCIPAIS

us-central1 (Iowa)
europe-west1 (Belgium)
asia-southeast1 (Singapore)
southamerica-east1 (São Paulo)
e outras

DESTAQUES

- API Cassandra (CQL)
- Escalabilidade automática
- Alta disponibilidade
- Backups e PITR
- Integração com IAM, VPC e Cloud Monitoring

 cloud.google.com/datastore



Microsoft Azure

Azure Managed Instance for Apache Cassandra

Serviço gerenciado baseado
em instâncias do Cassandra.

REGIÕES PRINCIPAIS

East US
West Europe
Southeast Asia
Brazil South
e outras

DESTAQUES

- Compatível com Cassandra OSS (CQL)
- Escalabilidade manual
- Alta disponibilidade
- Backups automáticos
- Integração com Azure AD, VNet e Monitor

 azure.microsoft.com/cassandra

DataStax Astra

DataStax Astra DB (for Apache Cassandra)

DBaaS nativo da DataStax,
criadores do Cassandra.

REGIÕES PRINCIPAIS

US East
US West
Europe
Asia Pacific
e outras

DESTAQUES

- 100% compatível com Cassandra
- Escalabilidade automática
- Multi-cloud (AWS, GCP, Azure)
- Alta disponibilidade
- Backups, segurança e métricas integradas

 astra.datastax.com



IBM Cloud

IBM Cloud Databases for Apache Cassandra

Serviço gerenciado baseado
em Cassandra open source.

REGIÕES PRINCIPAIS

US South
US East
Europe
Asia Pacific
e outras

DESTAQUES

- Compatível com Cassandra OSS (CQL)
- Alta disponibilidade
- Backups automáticos
- Integração com IAM, VPC e Monitoramento

 cloud.ibm.com/databases/cassandra



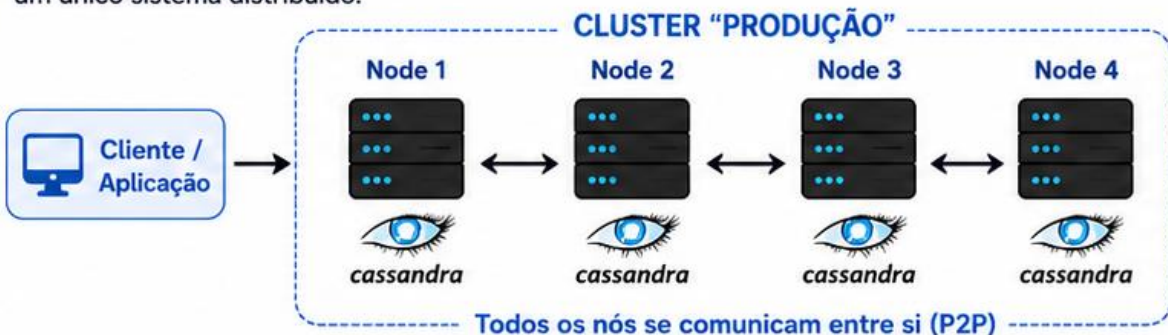
Todos os serviços oferecem compatibilidade com CQL
e permitem migrar aplicações que já usam Apache Cassandra.



Dica: Verifique sempre as regiões disponíveis, características e preços atualizados diretamente no site do provedor.

1. O QUE É UM CLUSTER CASSANDRA?

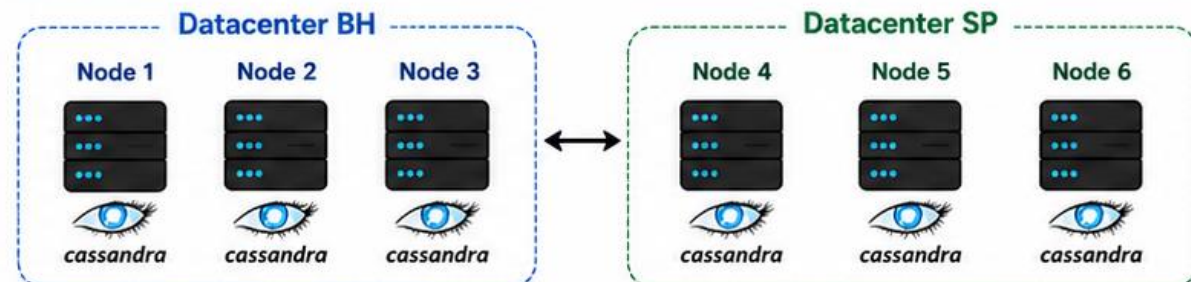
Um cluster Cassandra é um conjunto de servidores (nós) executando Cassandra e trabalhando juntos como um único sistema distribuído.



- Todos os nós são iguais (não existe master).
- Qualquer nó pode receber leitura ou escrita.
- Dados são distribuídos e replicados automaticamente.

2. DATACENTERS

Nós podem estar em locais diferentes (datacenters/regiões). Ainda assim, fazem parte do mesmo cluster.



Exemplo de criação de Keyspace com replicação por datacenter:

```
CREATE KEYSPACE saeweb
WITH replication = {
  'class' : 'NetworkTopologyStrategy',
  'BH' : 3,
  'SP' : 2
};
```

Significa que cada dado terá:
3 cópias em BH
+
2 cópias em SP
= 5 cópias no total

3. KEYSPACE E REPLICAÇÃO

Keyspace define a política de replicação: quantas cópias de cada dado existirá e em quais datacenters.



REGRAS IMPORTANTES

- ✓ Replication Factor (RF) **NÃO** cria nós. Ele apenas define quantas cópias serão armazenadas nos nós existentes.
- ✓ RF não pode ser maior que a quantidade de réplicas disponíveis para a estratégia utilizada.
- ✗ Com **NetworkTopologyStrategy**, a soma dos RFs por datacenter não pode ser maior que o total de nós disponíveis naquele cluster.

EXEMPLO:

Cluster com 4 nós (1 datacenter).
Tentando criar RF = 10



- ✗ Não é permitido. Existem apenas 4 nós para armazenar as cópias.

4. CONFIGURAÇÕES COMUNS

Quantidade de Nós	Replication Factor (RF)	Tolerância a falhas	Observação
1 nó	1	0 nós	Apenas desenvolvimento/testes
3 nós	3	1 nó	Mínimo recomendado para produção
4 nós	3	1 nó	Bom equilíbrio entre custo e HA
6 nós	3	2 nós	Ambiente mais robusto
9 nós ou mais	3	2 nós	Aumenta capacidade e desempenho
N nós	N-1	N-2 nós	Máxima redundância (custo alto)



BOAS PRÁTICAS

- Use **NetworkTopologyStrategy** em ambientes com múltiplos datacenters.
- Use RF = 3 na maioria dos casos (bom equilíbrio entre HA e custo).
- Monitore o cluster e adicione nós conforme o crescimento dos dados e das operações.

Keyspace: chat

Como os dados são organizados no Cassandra?

Table: users

Row: user_id = 101

- name: Evan
- email: evan@example.com
- age: 15

Row: user_id = 102

- name: Stefan
- email: stefan@example.com
- age: 25
- address: 2436 Data St, Seattle WA

Table: messages

Row: msg_id = 5

- sender: Evan
- text: "I love cassandra"

Row: msg_id = 6

- sender: Evan
- text: "Check out hellointerview.com!"

Um **Keyspace** no Apache Cassandra é o **contêiner de nível superior que agrupa tabelas relacionadas**, funcionando de forma análoga a um banco de dados em sistemas relacionais (RDBMS)

Keyspace

Equivalente a um banco de dados.

Também define:

- Fator de replicação, Estratégia de replicação, Data centers

CASSANDRA: PARTITION KEY EXPLICADA

O QUE É A PARTITION KEY?

A Partition Key define **em qual nó do cluster** os dados serão armazenados.



Ela é usada para calcular um hash, gerar um token e localizar o nó responsável dentro do ring.

EXEMPLO DE TABELA

```
CREATE TABLE user_messages (  
  user_id UUID,  
  message_id TIMEUUID,  
  text TEXT,  
  PRIMARY KEY (user_id, message_id)  
);
```

■ Partition Key = user_id
■ Clustering Key = message_id

INSERINDO REGISTROS

1 INSERT INTO user_messages
(user_id, message_id, text)
VALUES
('123', 'msg001', 'Olá');

hash(123) → Token → Ring → Node 3

Resultado no Node 3

user_id = 123
• msg001

2 INSERT INTO user_messages
(user_id, message_id, text)
VALUES
('123', 'msg002', 'Tudo bem?');

hash(123) → Token → Ring → Node 3

Resultado no Node 3

user_id = 123
• msg001
• msg002

→ Mesma Partition Key (123) → vai para o **MESMO** nó (Node 3)

3 INSERT INTO user_messages
(user_id, message_id, text)
VALUES
('456', 'msg003', 'Oi');

hash(456) → Token → Ring → Node 1

Resultado no Node 1

user_id = 456
• msg003

→ Partition Key diferente (456) → pode ir para **OUTRO** nó (Node 1)



POR QUE ISSO É IMPORTANTE?

Para consultas por Partition Key, o Cassandra lê **APENAS** o nó onde os dados estão.



Banco Relacional

Índice encontra os dados em qualquer lugar.



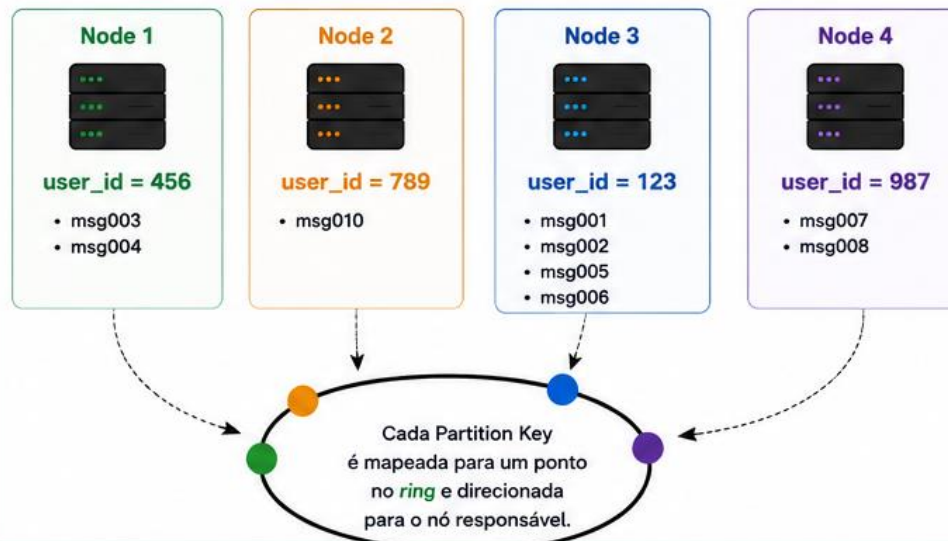
Cassandra



Partition Key encontra o nó onde os dados estão.

VISUALIZAÇÃO DO CLUSTER

Exemplo de distribuição de dados por Partition Key (user_id)



BOA PARTITION KEY DEVE:

- ✓ Distribuir os dados entre os nós do cluster
- ✓ Evitar partições gigantes (Hot Partitions)
- ✓ Ser utilizada nas consultas mais frequentes

EXEMPLO DE BOA MODELAGEM

```
CREATE TABLE logs_por_mes (  
  empresa_id INT,  
  ano_mes TEXT,  
  data TIMESTAMP,  
  descricao TEXT,  
  PRIMARY KEY ((empresa_id, ano_mes), data)  
);
```



Cada (empresa_id, ano_mes) vira uma partição diferente. Distribui os dados e evita partições gigantes.

O ERRO CLÁSSICO: PARTIÇÃO GIGANTE (HOT PARTITION)

Exemplo ruim:

```
CREATE TABLE logs (  
  empresa_id INT,  
  data TIMESTAMP,  
  descricao TEXT,  
  PRIMARY KEY (empresa_id)  
);
```

✗ Se empresa_id = 1 possui 500 milhões de logs, TUDO vai para o mesmo nó (ex.: Node 2). Isso causa **Hot Partition** e degrada o desempenho.

Resultado: todos os dados no mesmo nó



REGRA DE OURO

Para boa Partition Key = distribuição + consultas eficientes + partições equilibradas

Partition Key = endereço dos dados dentro do cluster.

Como Cassandra sabe em qual nó salvar um dado?



Consistent Hashing

O **Hashing Consistente** é a tecnologia que permite ao Cassandra **distribuir dados entre os nós do cluster sem precisar mover tudo quando um novo servidor é adicionado.**

Consistent Hashing

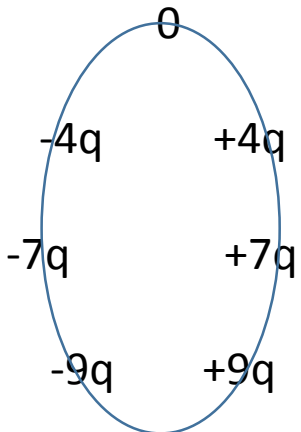
O Cassandra usa o algoritmo

Murmur3Partitioner por padrão para aplicar a função hash na Partition Key

Ele produz um inteiro de 64 bits.

-2^{63} até $2^{63} - 1$

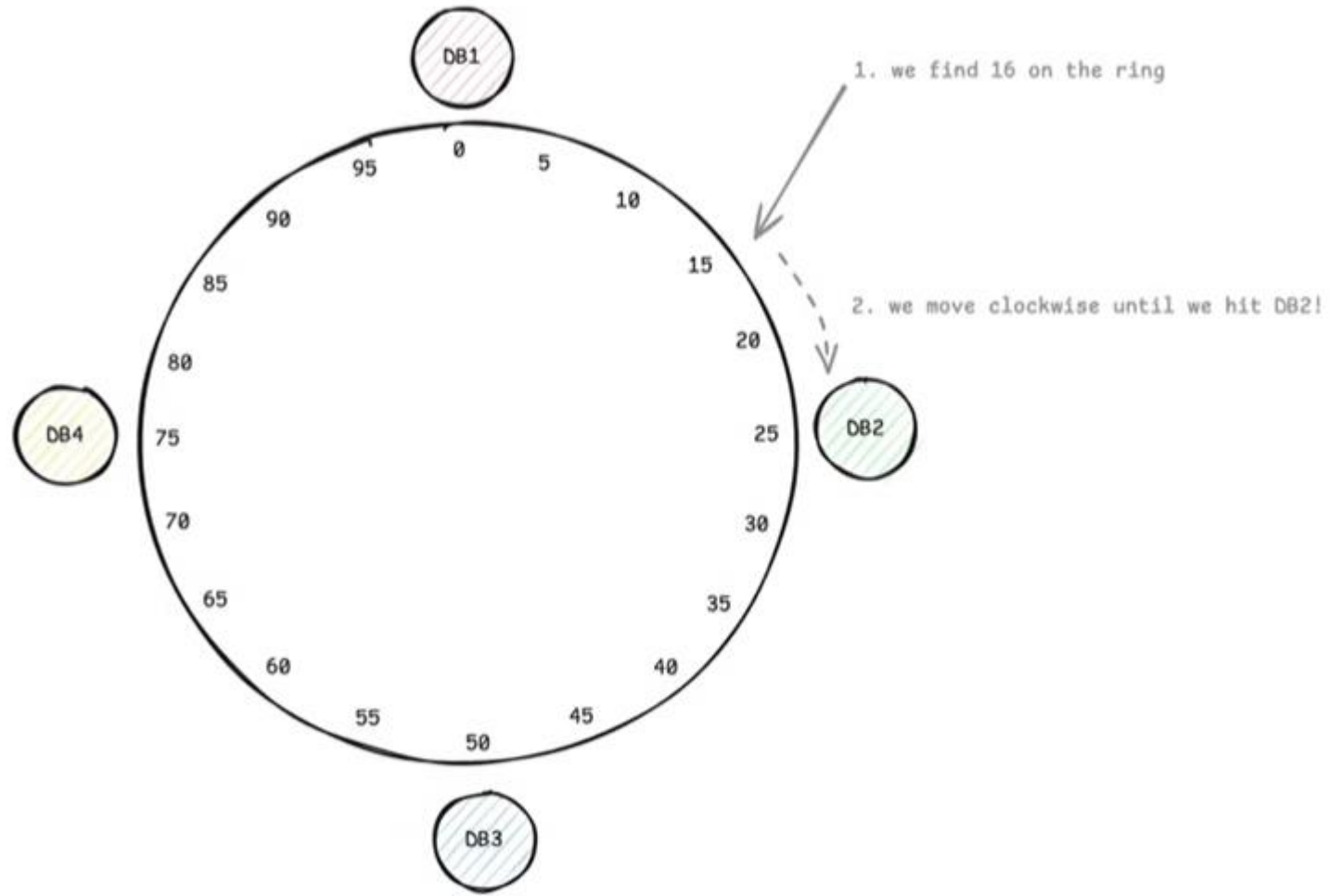
-9.223.372.036.854.775.808
até
+9.223.372.036.854.775.807



Token Ring

-9 quintilhões ----- +9 quintilhões

Por que números tão grandes?



Where database does event 2 live on?
Well, $\text{hash}(2) = 16$
so...

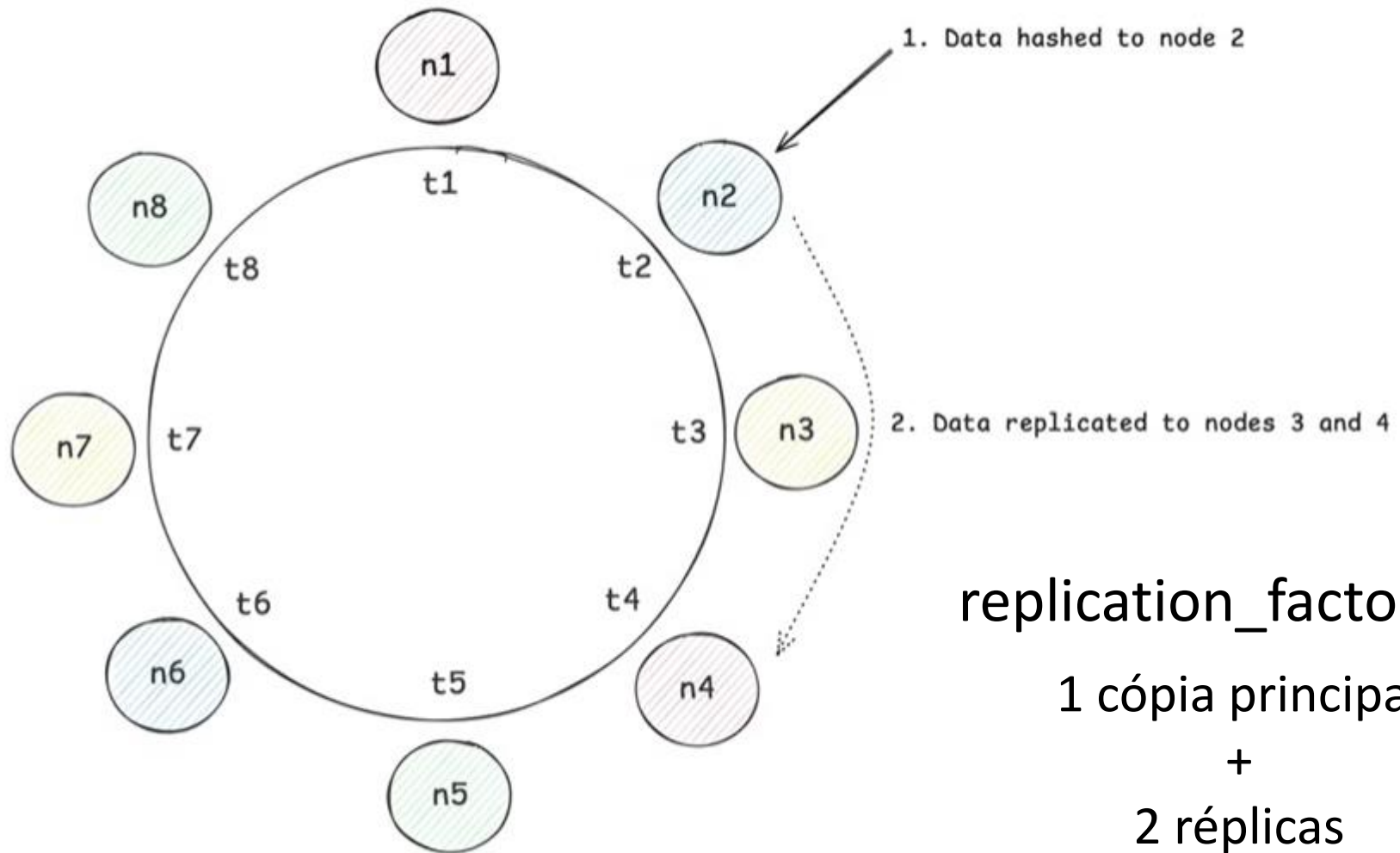
1. we find 16 on the ring

2. we move clockwise until we hit DB2!



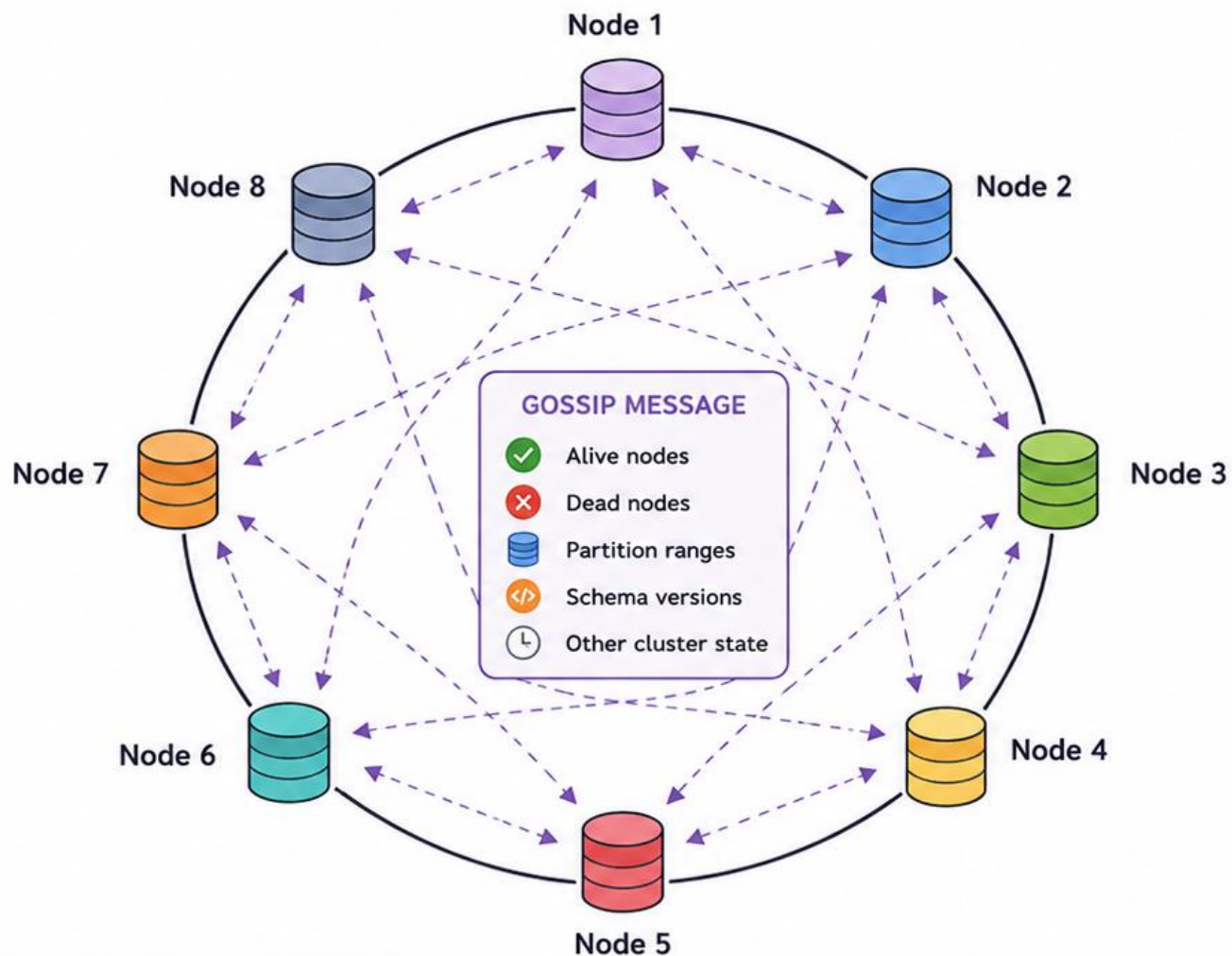
Para distribuir melhor os dados e evitar colisões!

Consistent Hashing



GOSSIP PROTOCOL

Nós trocam informações constantemente



↔ = Troca de mensagens Gossip
— = Token Ring (anel de tokens)

RESULTADO

Todos os nós têm uma visão consistente do cluster e podem detectar falhas rapidamente, sem um ponto único de falha.



ALIVE NODES

Cada nó informa quais nós estão ativos.



DEAD NODES

Se um nó falhar, os outros são avisados rapidamente.



PARTITION RANGES

Cada nó informa quais faixas do token ring ele é responsável.



SCHEMA VERSIONS

Todos os nós compartilham as versões do schema.



OUTROS ESTADOS

Configurações, load, capacidades e outras informações do cluster.

COMO FUNCIONA

Periodicamente, cada nó escolhe aleatoriamente outros nós e envia suas informações (gossip). Essas informações se espalham por todo o cluster de forma rápida e eficiente, sem necessidade de um servidor central.

CAP Theorem

C: Consistency

A: Availability

P: Partition Tolerance

Tunable Consistency

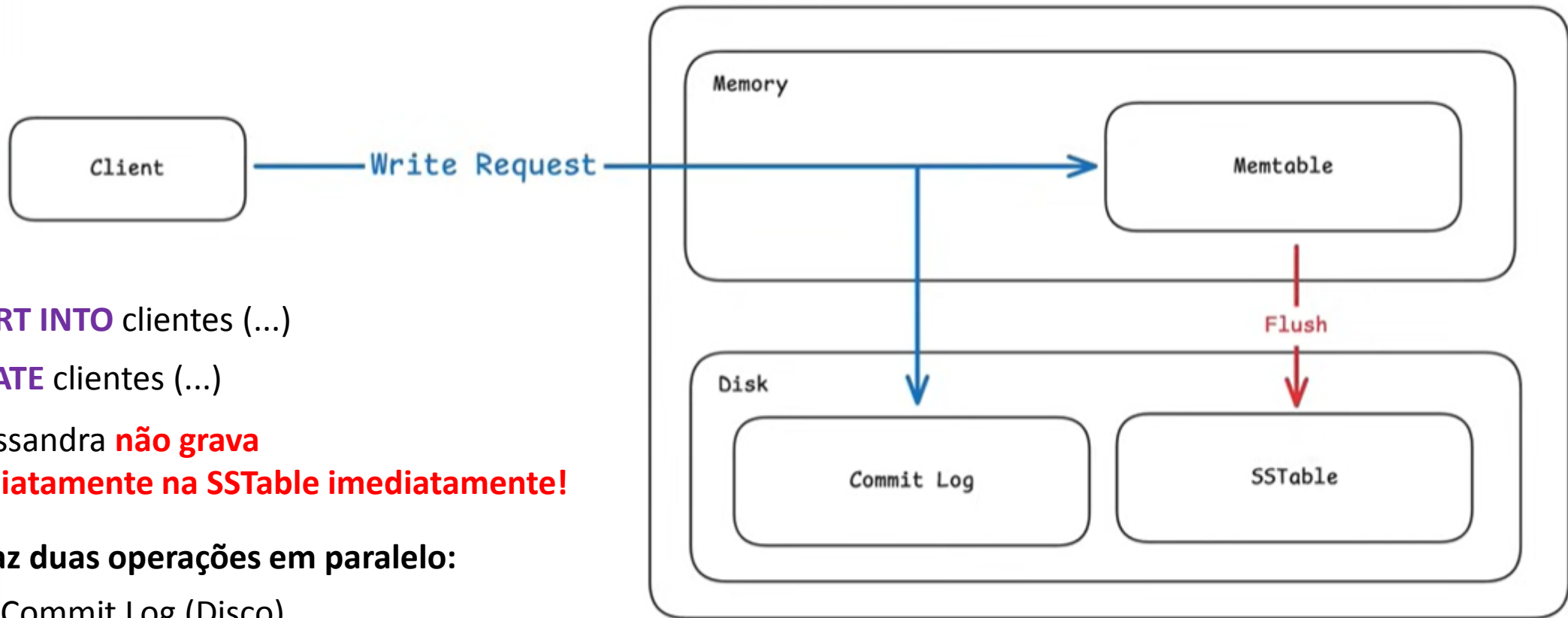
ONE waits for one replica – fastest, but weakest consistency.

QUORUM waits for a majority – a balanced middle ground.

ALL waits for every replica – strongest consistency, but slowest.

Escrita de dados no Cassandra

Cassandra Node



INSERT INTO clientes (...)

UPDATE clientes (...)

O Cassandra **não grava imediatamente na SSTable imediatamente!**

Ele faz duas operações em paralelo:

1. Commit Log (Disco)
2. Memtable (RAM)

Quando a **Memtable** atinge determinado tamanho:

- 256 MB
- 512 MB
- 1 GB

(depende da configuração)



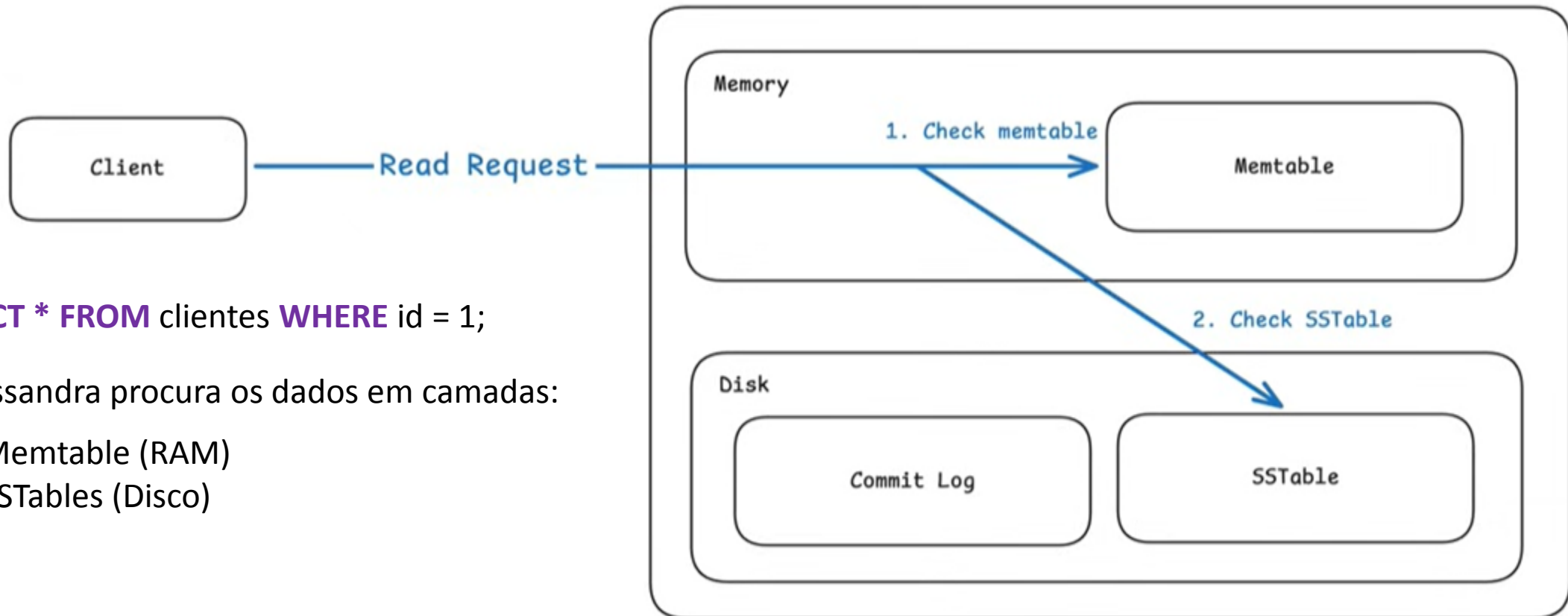
Flush

É a transferência:

Memtable
↓
SSTable

Sorted String Table

Leitura de dados no Cassandra



SELECT * FROM clientes **WHERE** id = 1;

O Cassandra procura os dados em camadas:

1. Memtable (RAM)
2. SSTables (Disco)

Por que, primeiro o Cassandra procura o dado na memória?

Porque os dados mais recentes ainda podem não ter sido gravados na SSTable.



**Flush não
aconteceu!**

Leitura de dados no Cassandra



SELECT * FROM clientes **WHERE** id = 1;

O Cassandra procura os dados em camadas:

1. Memtable (RAM)
2. SSTables (Disco)

Durante uma leitura, o Cassandra procura primeiro na **Memtable**, onde estão os dados mais recentes em memória. Caso necessário, consulta as **SSTables** em disco, utilizando estruturas como **Bloom Filters e índices** para localizar rapidamente os registros sem precisar varrer todos os arquivos.

Essa abordagem permite combinar alta velocidade de escrita com leituras eficientes em grandes volumes de dados.

Por que, primeiro o Cassandra procura o dado na memória?

Porque os dados mais recentes ainda podem não ter sido gravados na SSTable.

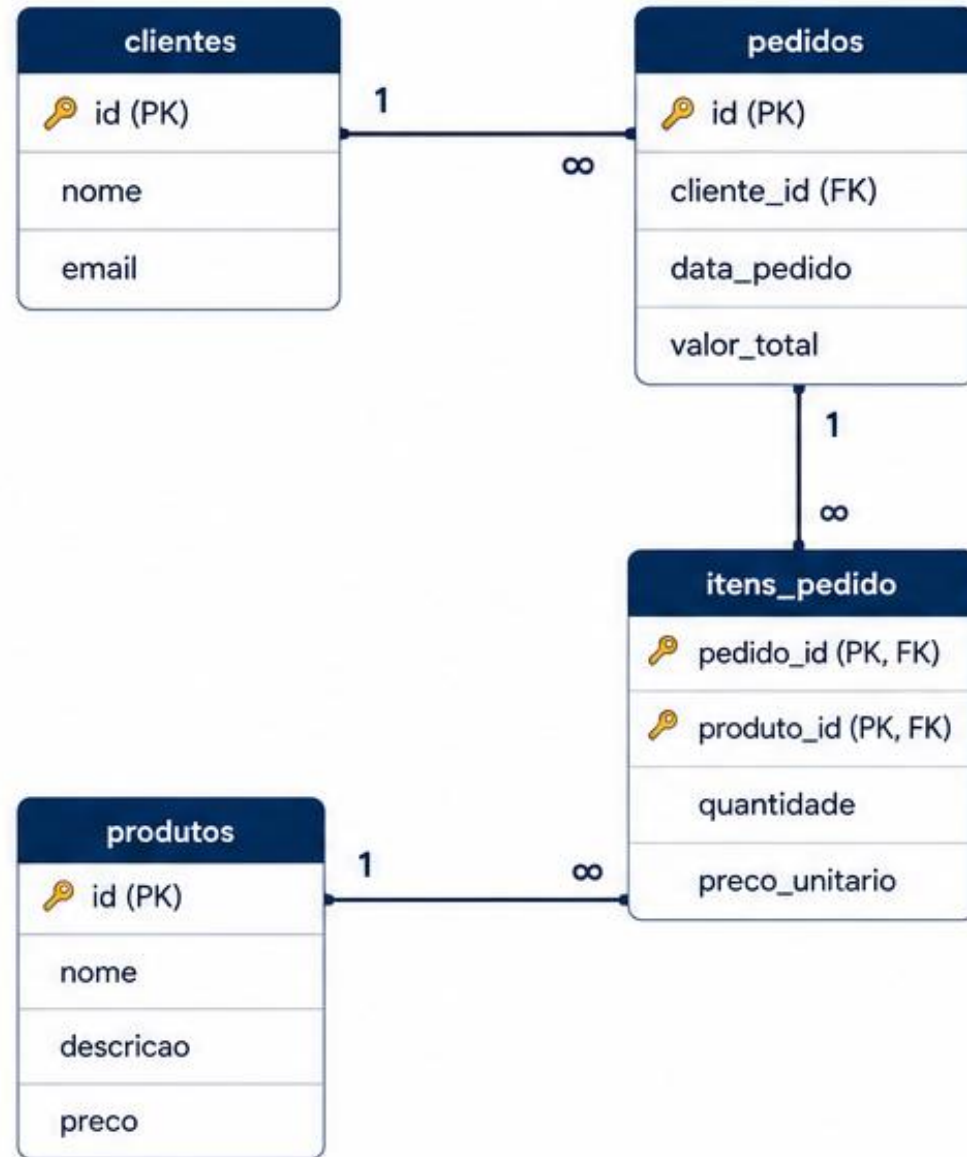


Flush não
aconteceu!

- ❑ O particionamento e o **hashing consistente** distribuem os dados entre os nós do cluster, permitindo que o Cassandra cresça facilmente.
- ❑ A **replicação** garante que existam **cópias dos dados em múltiplos nós**, aumentando a disponibilidade e a tolerância a falhas.
- ❑ O **protocolo Gossip** e a **arquitetura sem líder** mantêm o cluster descentralizado, resiliente e capaz de se recuperar automaticamente.
- ❑ A **consistência ajustável** permite definir, para cada operação, o equilíbrio desejado entre consistência dos dados e disponibilidade do sistema.



Como funciona a **modelagem**
de dados no Cassandra?



Em banco de dados relacionais....



- Dados divididos em várias tabelas
- Redundância mínima
- Integridade referencial
- Consultas usam JOINS para reunir os dados

EXEMPLO DE CONSULTA

```
SELECT c.nome, p.id, p.data_pedido, pr.nome,  
ip.quantidade, ip.preco_unitario  
FROM clientes c  
JOIN pedidos p ON p.cliente_id = c.id  
JOIN itens_pedido ip ON ip.pedido_id = p.id  
JOIN produtos pr ON pr.id = ip.produto_id  
WHERE c.id = 1;
```



No modelo relacional, os dados são organizados para minimizar redundância e garantir integridade, usando relacionamentos e JOINS.

Em banco de dados relacionais....

1) Tabela: pedidos_por_cliente

Consulta: listar pedidos e itens de um cliente

cliente_id	pedido_id	data_pedido	valor_total	cliente_nome	produto	quantidade	preco_unitario
1	1001	2025-05-15 10:30:00	350.00	João Silva	Notebook	1	300.00
1	1001	2025-05-15 10:30:00	350.00	João Silva	Mouse	2	25.00
...	...	...	...	...	...	...	...

Mesmo pedido
e cliente
repetidos
para cada
item

2) Tabela: pedidos_por_data

Consulta: listar pedidos de uma data (com cliente e itens)

data_pedido	pedido_id	cliente_id	cliente_nome	produto	quantidade	preco_unitario	valor_total
2025-05-15	1001	1	João Silva	Notebook	1	300.00	350.00
2025-05-15	1001	1	João Silva	Mouse	2	25.00	350.00
...	...	...	...	...	...	...	...

Dados do
pedido
repetidos
em outra
tabela

3) Tabela: pedidos_por_produto

Consulta: listar pedidos de um produto

produto	pedido_id	data_pedido	cliente_id	cliente_nome	quantidade	preco_unitario	valor_total
Notebook	1001	2025-05-15 10:30:00	1	João Silva	1	300.00	350.00
Mouse	1001	2025-05-15 10:30:00	1	João Silva	2	25.00	350.00
...	...	...	...	...	...	...	...

Pedido
repetido
novamente
para outra
consulta





- Dados reunidos em tabelas específicas para cada consulta
- Redundância intencional
- Sem JOINS
- Leituras muito rápidas, escritas mais custosas (atualizações em várias tabelas)
- Consistência eventual (BASE)

CONSULTA NO CASSANDRA (SEM JOIN)

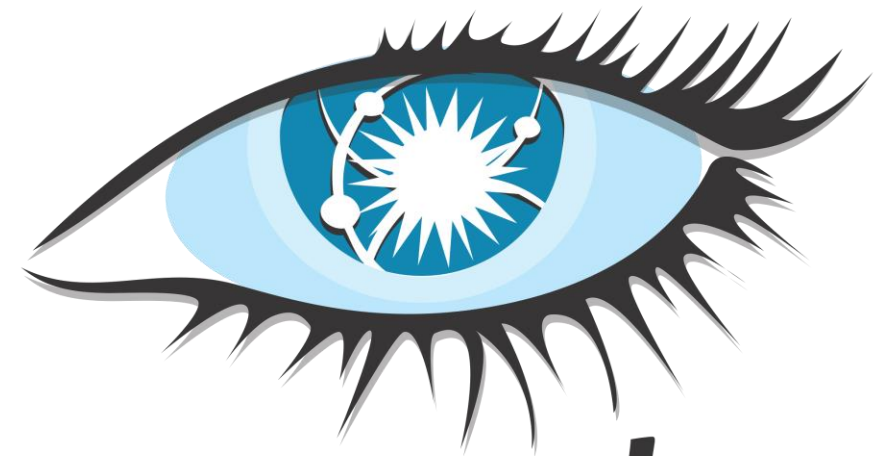
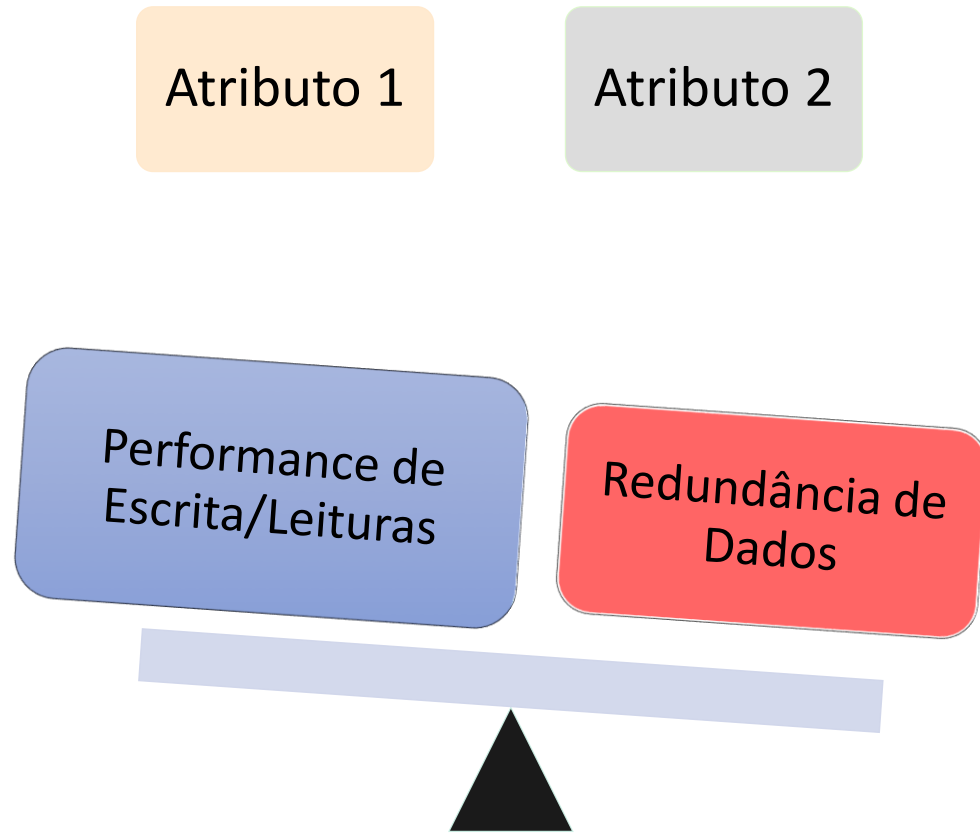
```
SELECT *  
FROM pedidos_por_cliente  
WHERE cliente_id = 1  
ORDER BY data_pedido DESC;
```



No Cassandra, não modelamos os dados.
Modelamos as consultas.



Trade-offs



cassandra

Use Cassandra

- High (100k+ rps) write throughput
- Write >> read
- Predictable, limit query pattern

DON'T use Cassandra

- Need flexible queries
- Need strong consistency

Conclusão

O Apache Cassandra é um banco de dados **NoSQL distribuído** projetado para armazenar grandes volumes de dados com alta disponibilidade, escalabilidade horizontal e tolerância a falhas. **Diferentemente dos bancos relacionais, ele modela os dados com base nas consultas esperadas, utilizando desnormalização e replicação para eliminar JOINS e maximizar o desempenho.** Sua arquitetura **sem servidor mestre**, combinada com particionamento e replicação automática dos dados, permite que o sistema continue operando mesmo diante da falha de nós, tornando-o uma excelente escolha para aplicações de larga escala, como redes sociais, IoT, monitoramento e sistemas de mensageria.

Referências

- ❑ https://cassandra.apache.org/_/index.html
- ❑ <https://www.ibm.com/br-pt/think/topics/cassandra>
- ❑ <https://www.youtube.com/watch?v=TD3-INhm60Q>